

Visuelle Programmierung mit vvvv – Basics

Was erwarte ich mir von der Praktikumsdokumentation/Protokoll?

- Beantworten der gestellten Fragen in einem eigenen Protokoll (Word)
- Speichern Sie alle „vvvv“-Patches ab, und geben Sie diese in Moodle ab (ZIP-Datei)
- Zusätzlich machen Sie bitte Screenshots, Fotos, etc. die Ihre Patches in „Aktion“ zeigen (diese können Sie ja direkt in „vvvv“ komfortabel über die Tastenkombinationen „STRG + 1“ bzw. „STRG + 2“ erstellen)
- Eigene Interpretation von Erkenntnissen des Experiments, Auffälligkeiten
- Welche Probleme sind Ihnen begegnet und wie haben Sie diese gelöst?

Wie?

- Antworten, Screenshots, Fotos, etc. direkt in Ihrem Word-Protokoll
- Patches als Upload (ZIP-Datei) in Moodle (Kursraum Technik der Mensch-Maschine Interaktion)
- Abzugeben sind also pro Gruppe 2 Dateien!

Abgabe?

- Fertigstellung des Protokolls und Upload der Patches bis spätestens zu folgenden Terminen:
 - TMIP.1: Sonntag, 10. November, 23:55 Uhr (Moodle)
 - TMIP.2: Montag, 4. November, 23:55 Uhr (Moodle)
 - TMIP.3/TMIP.4: Dienstag, 5. November, 23:55 Uhr (Moodle)

(Ab) heute beschäftigen wir uns mit der visuellen Programmiersprache bzw. -umgebung „vvvv“ (auch „v4“ genannt). „vvvv“ (www.vvvv.org/downloads) wurde mit dem Ziel implementiert, ein Werkzeug zur Entwicklung ansprechender Programme und Installationen für „Nicht-Programmierer“ zur Verfügung zu stellen, da Interaktionsdesigner oft abstrakten Programmcode nicht „verstehen“ – optimal also auch für UXD'ler ;-)

„vvvv“ erlaubt demnach die Entwicklung von Prototypen auf bildlicher Ebene (Programmieren mit Komponenten, verbinden der Komponenten mit „wires“, etc.). „vvvv“ ist frei verfügbar und die Community rund um „vvvv“ sehr groß – sodass Hunderte von Interaktionskomponenten (sogenannte „nodes“ zur Verfügung stehen, die in eigenen Programmen wiederverwendet werden können, z.B. zur Interaktion mit der Kinect, Leap Motion, etc. etc.).

Bitte beachten Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Einführung in vvvv“ (sowie die zugehörigen vvvv-Patches), die die Basisinformation zu vvvv, Grundbegriffe, erste Programme, etc., in einer Schritt-für-Schritt Anleitung (Tutorial-Stil) erläutern. Zusätzlich sei auf folgende Ressourcen hingewiesen, die Ihnen den Umgang mit „vvvv“ erleichtern können:

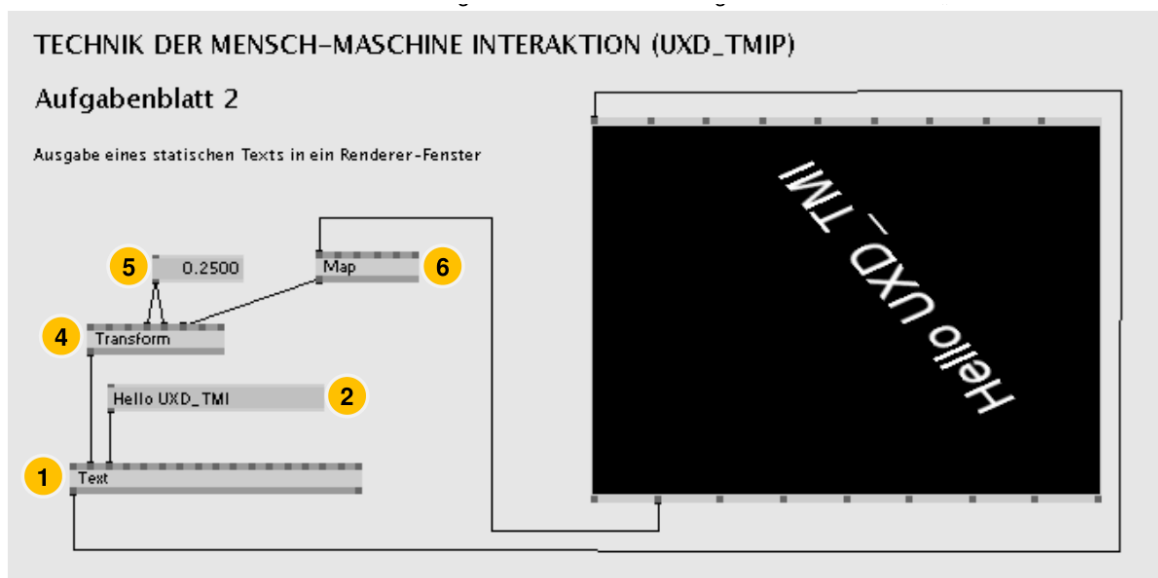
- Tutorial (ähnlich wie jenes der Vorlesung): [vvvvBasics - Tutorial in simple steps \(English\).pdf](#)
- Übersicht über die Tastatur- und Mausbelegung: [Leaflet dont panic - The noobs guide to vvvv.pdf](#)

1) Erste Schritte mit „vvvv“

Zunächst beginnen wir mit 2 „vvvv-Programmen“ (= „Patches“) aus dem „vvvv“-Tutorial der Vorlesung, um uns einen ersten Überblick über die Funktionsweise und das Verhalten von „vvvv“ zu verschaffen.

1.a) „Hello World“

Dieser „vvvv“-Patch gibt zunächst nur statischen Text aus. Wir fügen dann dynamisches Verhalten hinzu, sodass der Text schließlich zu rotieren beginnt. Eine Beschreibung finden Sie auch im „vvvv“-Tutorial.



Vorgangsweise

- Starten Sie „vvvv“ und legen Sie einen neuen, leeren Patch an

- Nun fügen Sie nach und nach die einzelnen Komponenten („Nodes“) wie folgt ein:

Hinweis: Die Nummernangaben in Klammern entsprechen den jeweiligen Zahlen in den Kreisen der Patch-Screenshots

(1) Doppelklick (links) in das Fenster öffnet den „Node-Inspector“ – hier sind alle verfügbaren „Nodes“ (Komponenten) aufgelistet, über das Eingabefeld kann gefiltert werden. Suchen Sie hier den „Text (EX9)“-Node und fügen ihn in Ihren Patch ein.

(2) Fügen Sie nun einen Node vom Typ „IOBox (String)“ ein, Doppelklicken Sie mit der linken Maustaste auf den Node und geben Sie den Text „Hello World“ ein. Sie können den Node vergrößern, damit Sie auch den gesamten eingegebenen Text im Patch sehen. Verbinden Sie den Output-Pin dieses Nodes (2) mit dem 3. Eingabe-Pin des „Text“-Nodes (1)

(3) Fügen Sie einen „Renderer (EX9)“-Node ein (es öffnet sich ein neues DirectX Renderer-Fenster) und verbinden Sie den Output-Pin des „Text“-Nodes (2) mit dem 1. Eingabe-Pin des „Renderer“-Nodes (3). Selektieren Sie den „Renderer“-Node und drücken „ALT + 2“ um den Renderer direkt in den Patch zu integrieren. Vergrößern Sie das „Renderer“-Fenster und speichern Sie den Patch als „HelloWorld.v4p“ ab. Sie bemerken auch, dass in diesem Fenster bereits der eingegebene Text angezeigt wird - allerdings viel zu groß...

(4) Um den Text kleiner darzustellen, fügen Sie zunächst einen Node vom Typ „Transform (Transform 2D)“ ein und verbinden seinen Output-Pin mit dem 2. Eingabe-Pin des „Text“-Nodes (1)

(5) Fügen Sie einen „IOBox (Value Advanced)“-Node ein, Doppelklicken Sie darauf mit links und geben den Wert „0.25“ ein. Verbinden Sie diesen Node mit den Eingabe-Pins 4 („ScaleX“) und 5 („ScaleY“) des „Transform“-Nodes (4). Der Text wird in beiden Dimensionen (X, Y) im gleichen Maßstab skaliert

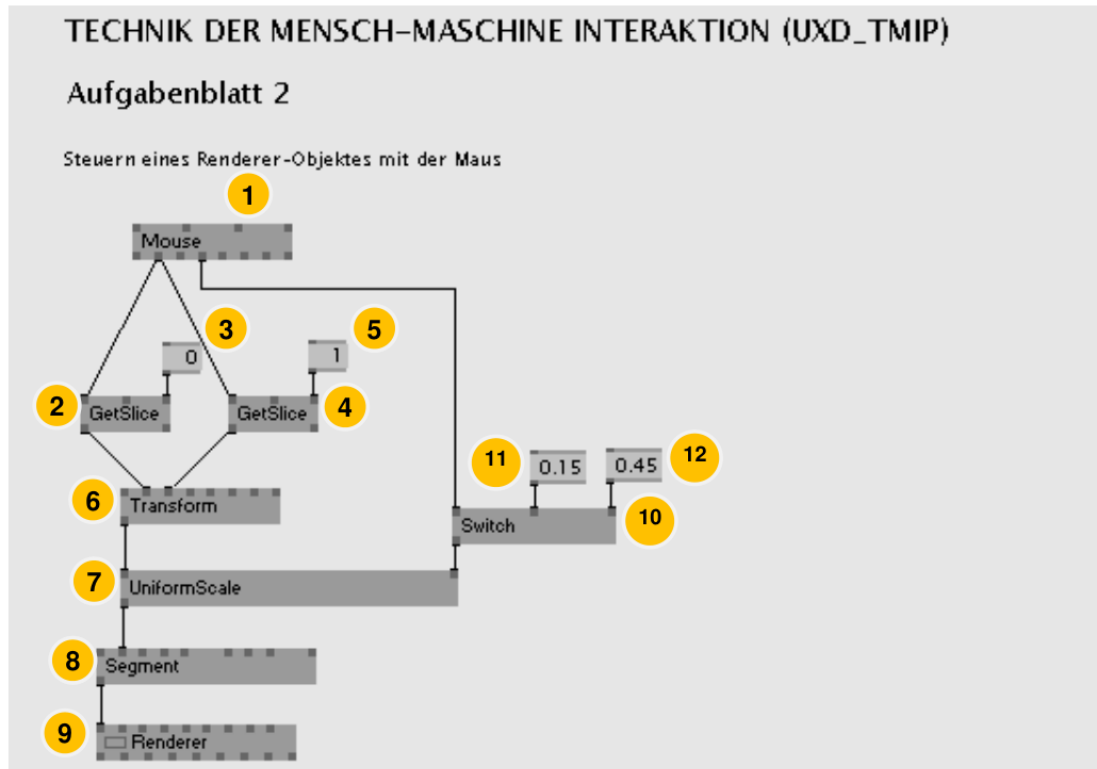
(6) Um nun den im Renderer-Fenster (3) angezeigten Text durch Mausbewegungen rotieren zu lassen, fügen Sie einen neuen Node vom Typ „Map (Value)“ ein. Den Eingabe-Pin dieses Nodes verbinden Sie mit dem 2. Ausgabe-Pin des Renderers (3), und den Ausgabe-Pin des „Map“-Nodes verbinden Sie mit dem 6. Eingabe-Pin (beschriftet mit „Rotate“) des „Transform“-Nodes (4).

- Speichern Sie den Patch mit Dateinamen „HelloWorldExt.v4p“ ab
- **Frage 1:** Wodurch wird die Drehung des Texts im Renderer-Window ausgelöst bzw. beeinflusst? Was ändert sich, wenn Sie anstatt dem 2. Output-Pin des „Renderer“-Nodes den 1. Output-Pin verwenden (und mit dem 1. Eingabe-Pin des „Map“-Nodes) verbinden?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.

1.b) Maussteuerung mit „vvvv“

Das 2. Beispiel finden Sie ebenfalls im „vvvv“-Tutorial und soll Ihnen vermitteln, wie einfach es in „vvvv“ ist, in beliebigen Patches auf externe Eingabegeräte zuzugreifen und damit interaktive Programme zu erstellen.



Was die einzelnen Komponenten genau machen, müssen Sie hier zunächst einmal noch **nicht verstehen** – führen Sie dieses „Mini-Tutorial“ einfach wie nachfolgend beschrieben Schritt-für-Schritt durch. Auch müssen Sie nicht darauf achten, dass die Verbindungslinien („wires“) gerade bzw. rechtwinkelig sind – dafür ist später noch genug Zeit... (Hinweis: mit „**STRG+Y**“ können sie zwischen Beziér-Verbindungslinien, rechtwinkelligen Verbindungen und „schrägen“ Direktverbindungen von Nodes wechseln).

Der oben dargestellte und nachfolgend beschriebene Patch erfüllt die gleiche Funktion wie jener im Tutorial, zeigt hier aber eine geringfügig abweichende Verarbeitung (Verwendung eines „Uniform-Scale“-Nodes) um zu verdeutlichen, dass es in „vvvv“ meist mehrere Möglichkeiten gibt, um ein Problem zu lösen.

Vorgangsweise

- Starten Sie „vvvv“ und legen Sie einen neuen, leeren Patch an

(1) Doppelklick (links) in das Fenster öffnet den „Node-Inspector“ – hier sind alle verfügbaren „Nodes“ (Komponenten) aufgelistet, über das Eingabefeld kann gefiltert werden. Suchen Sie hier den „Mouse (Devices Window)“-Node und fügen ihn in Ihren Patch ein.

- Für jeden Node gibt es im „Node-Inspector“ eine **Hilfeseite**. Ein **Klick mit der rechten Maustaste auf den ausgewählten Node** im „Node-Inspector“ öffnet die entsprechende Hilfeseite – probieren Sie das mal aus: Doppelklick auf den „Mouse“-Node → Auswahl „Mouse (Devices Window)“ mit der Maus → Rechtsklick. Die Hilfeseite sollte sich öffnen. Schließen Sie dieses Fenster wieder über das „vvvv“-Hauptmenü (mittlere Maustaste) → „Close“ im linken, oberen Fenster bzw. mit der Tastenkombination „STRG + W“. Weiters ist es möglich, die **Hilfeseite** eines markierten Nodes direkt im Patch über „**F1**“ aufzurufen.

- ◦ **Hinweis:** Für eine leichtere Konfigurierbarkeit der einzelnen „Nodes“ bietet es sich an, den „**Herrn Inspektor**“ („vvvv“-Hauptmenü über mittlere Maustaste öffnen, dort im linken, unteren Quadranten „Inspektor“ wählen bzw. Tastenkombination „STRG + I“) zu verwenden. Dieses Fenster sollten Sie während der Arbeit an Patches immer geöffnet halten.
- Neue/weitere Elemente fügen Sie nun mit einem Doppelklick links in das „vvvv“-Programmfenster ein. Für jedes eingefügte Element sehen Sie sich bitte a) die Hilfeseite an, b) die Ein-/Ausgänge die der „Node“ anbietet bzw. c) die erweiterten Konfigurationsmöglichkeiten im „Herrn Inspektor“. (Informationen zu den einzelnen Komponenten finden Sie auch im „vvvv“-Tutorial)
- (2) Node-Type „GetSlice (Spreads)“, mit dem „Mouse“-Node verbinden (wie oben dargestellt)
- (3) Node-Type „IOBox (Value Advanced)“, Value Type „Integer“, Default „0“ (null), mit dem „GetSlice“-Node verbinden
- (4) Node-Type „GetSlice (Spreads)“, mit dem „Mouse“-Node verbinden (wie oben dargestellt)
- (5) Node-Type „IOBox (Value Advanced)“, Value Type „Integer“, Default „1“, mit dem „GetSlice“-Node verbinden
- (6) Node-Type „Transform (Transform 2D)“, Pins 2 und 3 mit den beiden „GetSlice“-Nodes verbinden
- (7) Node-Type „UniformScale (Transform)“ und Verbinden mit dem „Transform“-Node
- (8) Node-Type „Segment (DX9)“, Verbindung mit dem „UniformScale“-Node
- (9) Node-Type „Renderer (EX9)“, es öffnet sich ein neues DirectX Renderer-Fenster; Renderer mit dem „Segment“-Node verbinden
- Patch als „Maussteuerung.v4p“ abspeichern
- Im Renderer-Fenster sollte nun bereits ein Objekt (in weiß) erscheinen welches Sie mit der Maus kontrollieren können... **Frage 2:** Was fällt Ihnen auf?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*
- Öffnen Sie den „Herrn Inspektor“ (falls nicht schon bereits geöffnet), wählen Sie den „Renderer“-Node (9) aus und ändern Sie den Wert des Attributes „Clear“ (rechte Maustaste) **Frage 3:** Was stellen Sie fest?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*
- Ändern Sie im „Herrn Inspektor“ den Wert „Resolution“ des „Segment“-Nodes (8) auf 50 – **Frage 4:** Was stellen Sie fest?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*
- **Frage 5:** Was passiert, wenn Sie die Ausgänge der beiden „GetSlice“-Nodes (2) und (4) vertauscht an den „Transform“-Node anschließen?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*
- Sie haben vermutlich auch schon festgestellt, dass sich Mausklicks (noch) nicht auf die Größe des weißen Kreises auswirken. Fügen Sie für dieses Verhalten die Komponenten (10), (11) und (12) ein:
- (10) Node-Type „Switch (Value Input)“, verbinden mit „Mouse“-Node „LeftButton“ und dem „UniformScale“-Node
- (11) Node-Type „IOBox (Value Advanced)“, verbinden mit dem „2. Eingang“ des „Switch“-Nodes (10), Doppelklick links auf den „IOBox“-Node und Eingabe des Wertes „0.15“

(12) Node-Type „IOBox (Value Advanced)“, verbinden mit dem „3. Eingang“ des „Switch“-Nodes (10), Doppelklick links auf den „IOBox“-Node und Eingabe des Wertes „0.45“

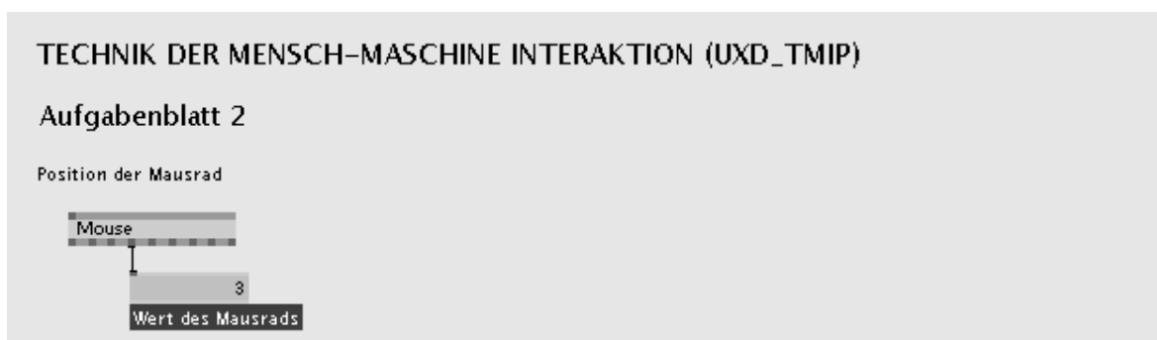
- **Frage 6:** Was bemerken Sie, wenn Sie die Werte der beiden „IOBox“-Nodes (11) und (12) ändern? Welche Werte sind überhaupt als Eingabe erlaubt bzw. sinnvoll?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*
- **Frage 7:** Was müssen Sie am Patch ändern, um die Größe des Kreises nicht mehr über die linke, sondern die mittlere Maustaste zu verändern? Wie müssen Sie den Patch verändern, dass das Objekt bei einem Mausklick „verschwindet“ (und sonst, wie bereits zuvor, mit der Mausbewegung verschoben wird)?
- *Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.*

2) Verwendung der Maus als eindimensionaler Positionsgeber

Wie haben bereits gesehen, dass man in „vvvv“ über den „Mouse (Devices Window)“-Node ganz einfach Zugriff auf die Maus bekommt. Es gibt noch einen zweiten Node für den Mauszugriff: „Mouse (Devices Desktop)“. Der einzige Unterschied zwischen den beiden Nodes ist, dass der erste die Position der Maus nur erfasst (und verarbeitet), wenn sich diese innerhalb eines Renderer-Fensters befindet. „Mouse (Devices Desktop)“ hingegen gibt ständig die aktuellen Positionswerte der Maus aus. Für das folgende Beispiel verwenden wir den Node „Mouse (Devices Window)“. Wie oben bereits gesehen, verfügt ein „Mouse (...)“-Node über mehrere Ausgabe-Pins. Neben der X- und Y-Position der Maus gibt jeder Node auch alle drei Tasten sowie die Position des Mausekkrads aus. Wir sehen uns nun an, wie (bzw. wozu) man die Position des Mausekkrads in einer eigenen Applikation verwenden könnte.

2.a) Darstellung der Position des Mausekkrads

Das Mausekkrad einer Maus lässt sich unendlich lange in eine Richtung drehen – der Wert des Mausekkrads nimmt damit beliebig große oder kleine Werte an. Je nachdem, wie lange in eine bestimmte Richtung gedreht wird, erhöht oder verringert sich der Wert des Mausekkrads beständig und weist kein Ende auf...



Vorgangsweise

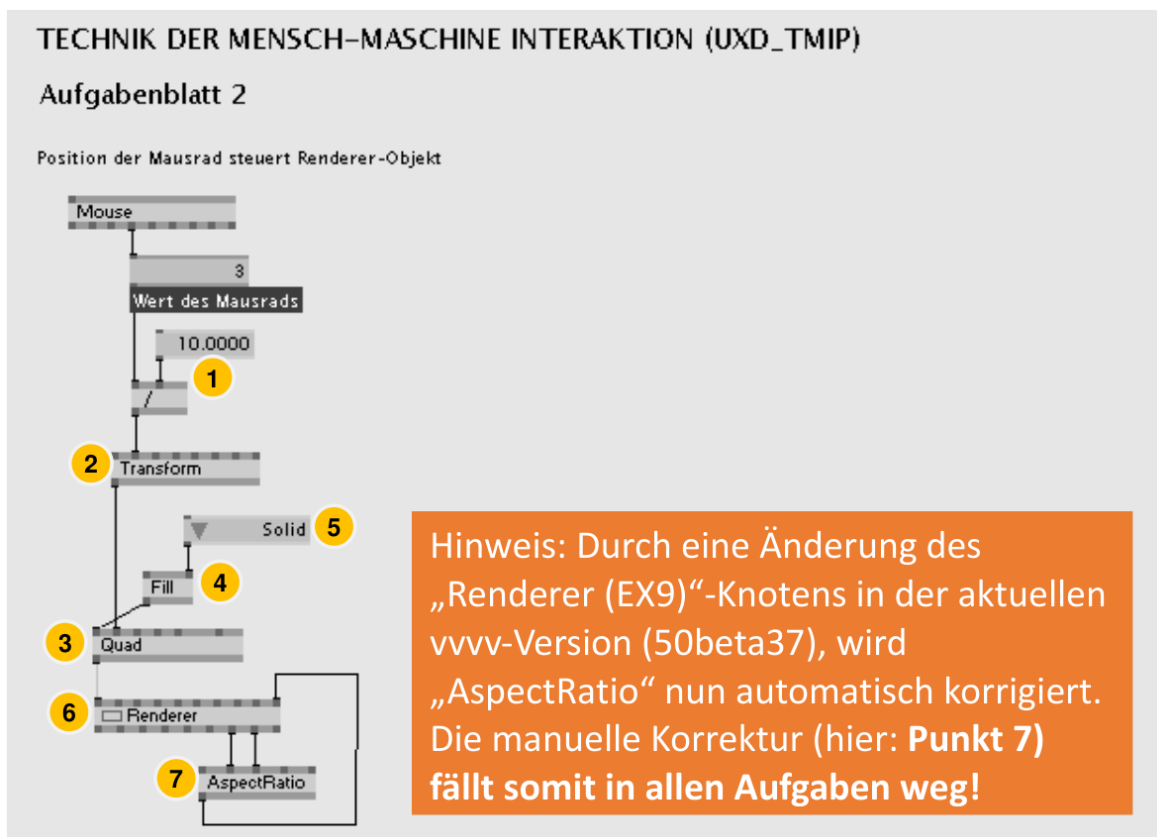
- Starten Sie „vvvv“ und legen Sie einen neuen, leeren Patch an
- Fügen Sie über den „Node-Inspector“ (Doppelklick links) einen „Mouse (Devices Desktop)“-Node in Ihren Patch ein
- Fügen Sie einen Node vom Typ „IOBox (Value Advanced)“ ein, setzen Sie den Value Type auf „Integer“ und verbinden Sie den Output-Pin des „Mouse (Devices Desktop)“-Nodes mit der Bezeichnung „Mouse Wheel“ mit dem

Input-Pin dieses Nodes. **Hinweis: Achten Sie immer auf die Bezeichnung der Pins eines Nodes – die Position (hier z.B. ganz links) kann, je nach Konfiguration, bei verschiedenen Rechnern unterschiedlich sein!**

- Speichern Sie den Patch als „Mausrad1.v4p“ ab und machen Sie (bei selektiertem Patch-Fenster) mit „STRG + 1“ bzw. „STRG + 2“ einen Screenshot Ihres Patches. *Kopieren Sie den Screenshot in Ihr Protokoll.*
- Wenn Sie nun das Mausrad drehen, zeigt die „IOBox“ den aktuellen Wert des Mausrads an. Wie Sie feststellen werden, kann der Wert beliebig groß oder klein werden... (Sie können den Wert auch im „Herrn Inspektor“ mitverfolgen; Selektieren der „IOBox“ nicht vergessen!)

Im nächsten Schritt erweitern wir den Patch etwas um den Mausrad-Wert zur Steuerung eines Objektes im DirectX-Renderer zu verwenden.

Hinweis: Durch eine Änderung des „Renderer (EX9)“-Knotens in der aktuellen vvvv-Version (50beta37), wird „AspectRatio“ nun automatisch korrigiert. Die manuelle Korrektur (hier: Punkt 7) fällt somit in allen Aufgaben weg!



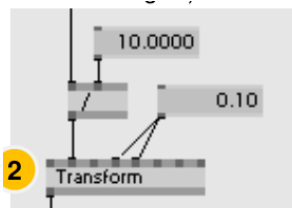
Vorgangsweise

- Als Basis verwenden wir den Patch „Mausrad1.v4p“ (und speichern ihn als „Mausrad2.v4p“ ab).
- Als allererstes ersetzen wir den Node „Mouse (Devices Desktop)“ durch einen Node „Mouse (Devices Window)“-Node. Dazu führen wir einen „Doppelklick links“ auf den bereits vorhandenen „Mouse“-Node aus, und wählen im „Node Inspector“ den Node „Mouse (Devices Window)“ aus. Nachdem die beiden Nodes die gleichen „Anschlüsse“ besitzen, bleiben die Verbindungen auch erhalten.

– Danach bauen wir die Schaltung Schritt-für-Schritt auf:

(1) Um die Sensitivität des Mausekners zu erhöhen, fügen wir einen Divisions-Node („/ (Value)“) sowie eine „IOBox (Value Advanced)“ ein und verbinden beide wie oben dargestellt. Den Wert der „IOBox“ stellen wir zunächst auf „10.0“ ein.

(2) Node-Typ „Transform (Transform 2D)“; Verbinden von Pin 2 mit dem Ergebnis-Pin des Divisionsoperators (1). Im „Herrn Inspektor“ die Werte für „ScaleX“ und „ScaleY“ jeweils auf 0.1 setzen (alternativ könnte man dazu auch „IOBox (Value Advanced)“-Nodes in den Patch einfügen):



(3) Node-Typ „Quad (DX9)“ zum Zeichnen eines Quadrats im Renderer-Fenster

(4) Einfügen eines Nodes vom Typ „Fill (EX9.RenderState)“

(5) Einfügen eines „IOBox (Enumerations)“-Nodes, Auswahl von Option „Solid“ und verbinden wie oben dargestellt

(6) Einfügen eines „Renderer (EX9)“-Nodes, es öffnet sich ein neues DirectX Renderer-Fenster; dieses Fenster, so wie unten dargestellt, möglichst breit ziehen

(7) Um dafür zu sorgen, dass das Quadrat im Renderer-Fenster nicht verzerrt wird, fügen wir nun noch einen Node vom Typ „AspectRatio (Transform)“ ein. Dessen Output-Pin wird mit dem Eingabe-Pin „AspectRatio“ des Renderers verbunden. Die beiden Output-Pins „Actual Backbuffer Width“ und „Actual Backbuffer Height“ des „Renderer (EX9)“-Nodes werden mit dem 2. und 3. Eingabe-Pin von „AspectRatio (Transform)“ verbunden

- Patch abspeichern
- Wenn Sie jetzt in das Renderer-Fenster klicken, können Sie mit dem Mausekner das Quadrat nach links und rechts verschieben...



- **Frage 8:** Welches Problem stellen Sie bei der „Navigation mit dem Mausekner“ fest? Ist das in dieser Form gewünscht bzw. welches Verhalten würden Sie sich für eine Anwendung im Normalfall wünschen?
- Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.

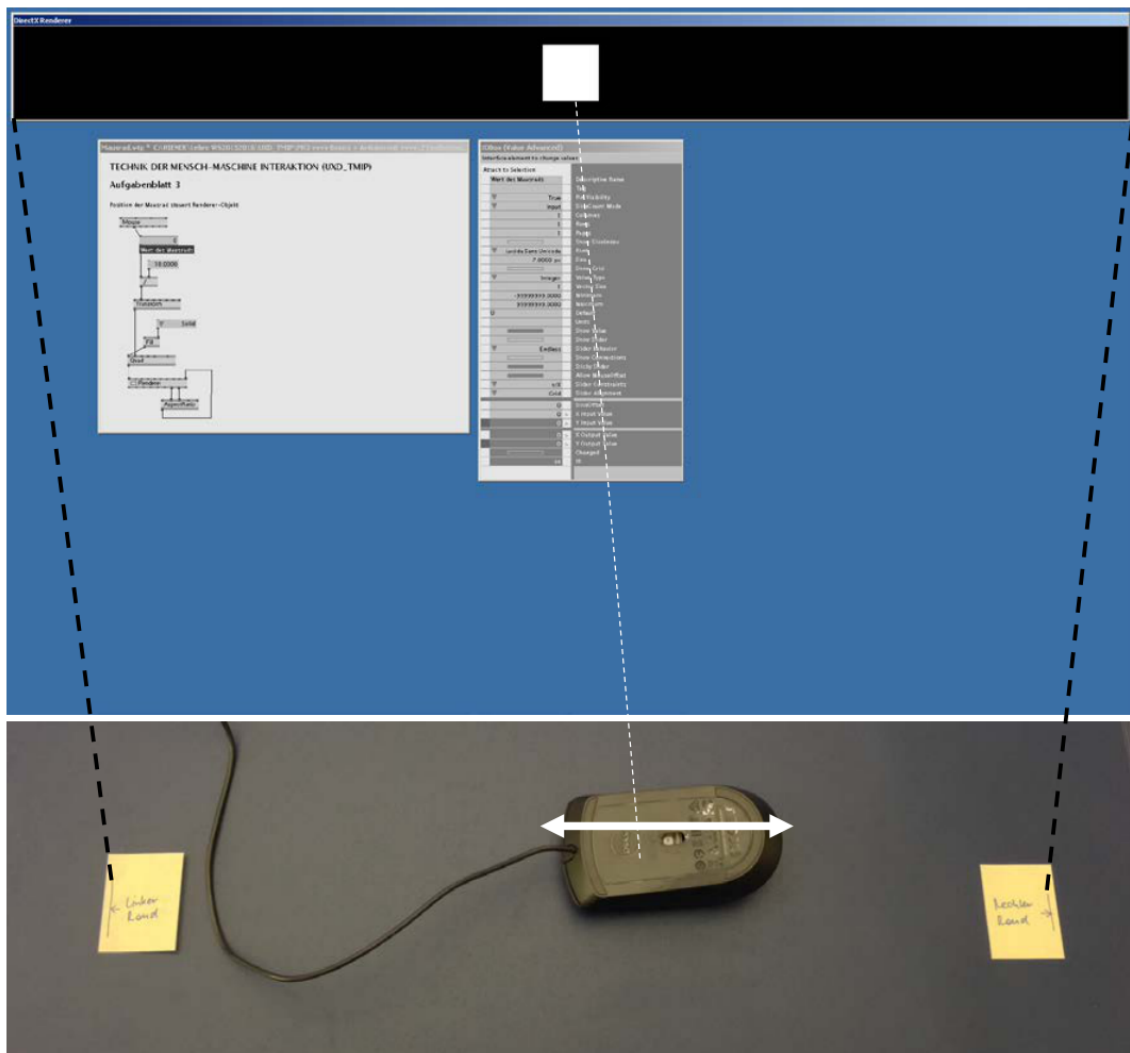
Wie soeben festgestellt erweist sich das Verhalten des Mausekners hier problematisch, da man für eine Positionsangabe immer darauf angewiesen ist, den aktuellen Wert der Maus und auch die möglichen Extreme zu kennen. Um die Position der Maus im aktuellen Fenster erfassen zu können, bauen wir nun in der nächsten Aufgabe eine kleine Hilfe in den Patch ein.

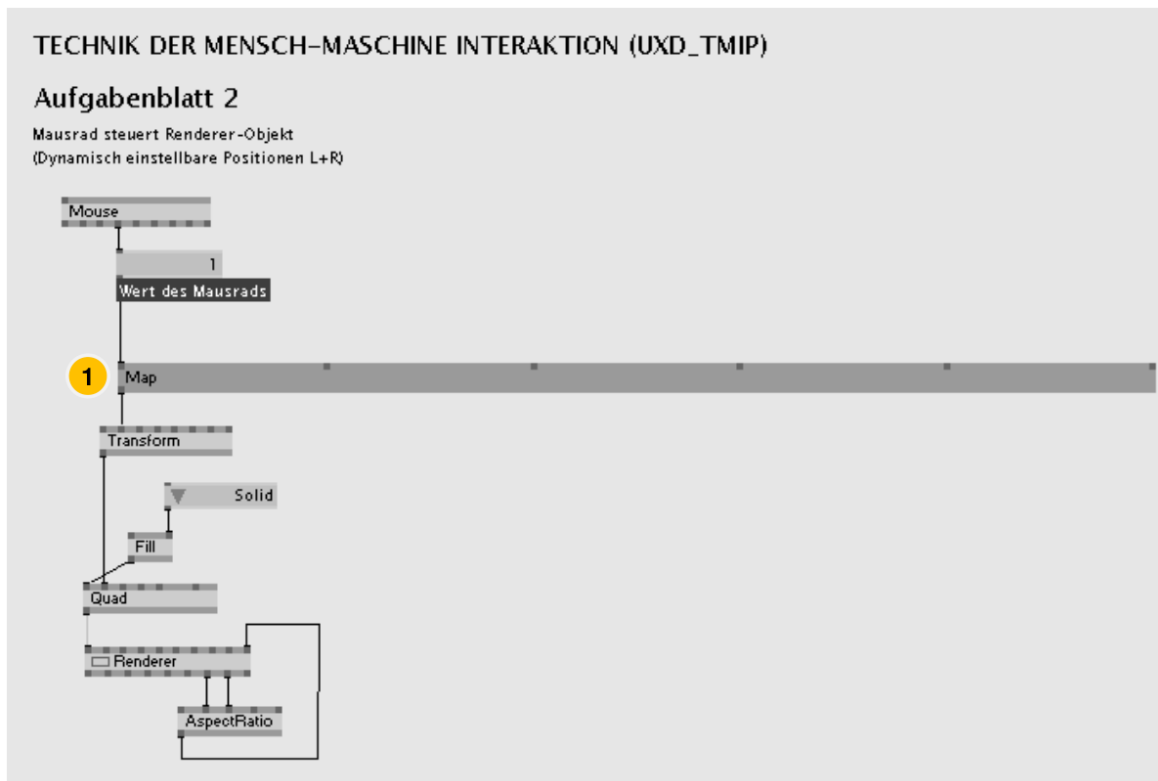
2.b) Dynamisch einstellbare Positionen für die Mausexploration

Das Mausexploration einer Maus lässt sich unendlich lange in eine Richtung drehen – der Wert des Mausexploration nimmt damit beliebig große oder kleine Werte an. Je nachdem, wie lange in eine bestimmte Richtung gedreht wird, erhöht oder verringert sich der Wert des Mausexploration beständig und weist kein Ende auf...

Die Idee wäre (wie in nachfolgender Skizze dargestellt), das Mausexploration der „umgedrehten“ Maus als eindimensionalen Positionsgeber zu verwenden. Die Markierungen „linker Rand“ und „rechter Rand“ am Schreibtisch sollen dabei dynamisch veränderbar sein, und der entsprechende Bereich so auf das Explorer-Fenster „gemappt“ werden, dass der „linke Rand“ am Schreibtisch mit dem linken Rand des Explorer-Fensters übereinstimmt (analog für den rechten Rand).

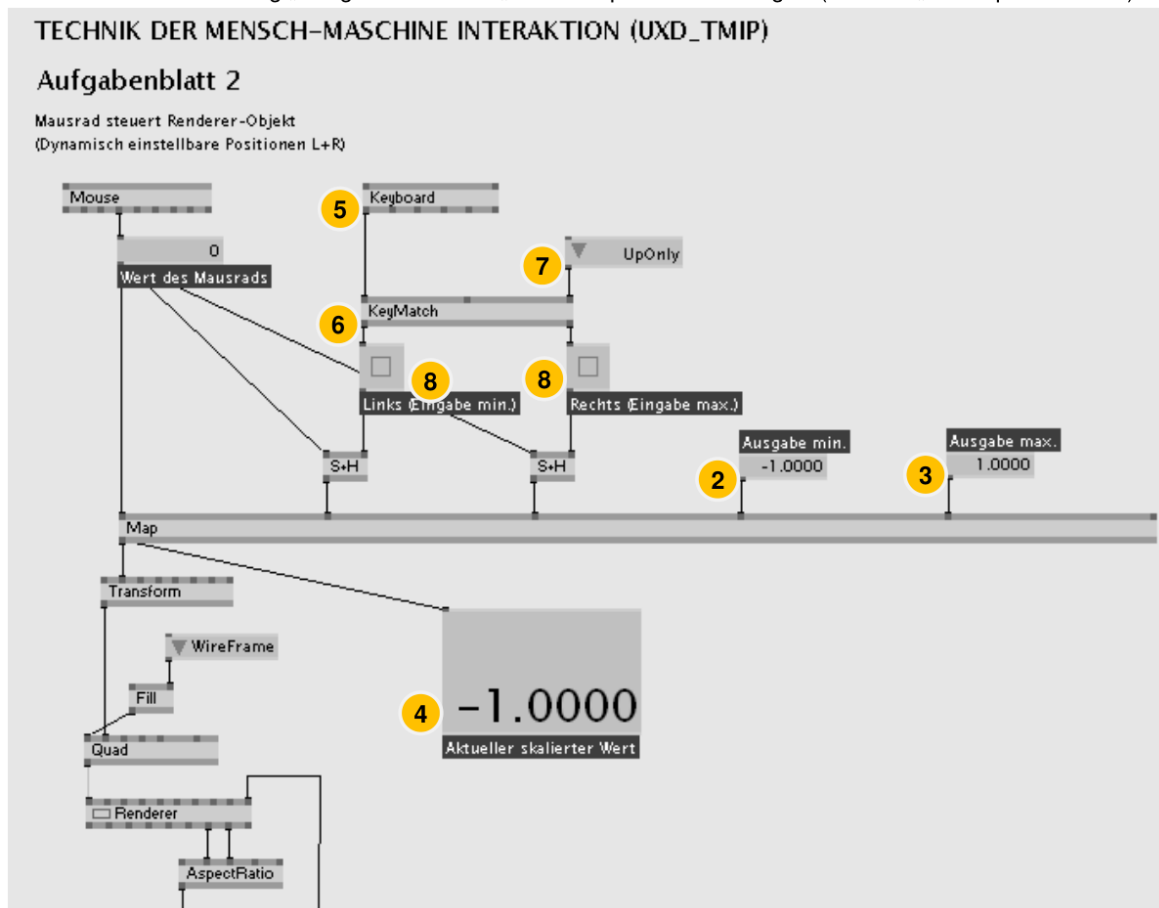
Für den nächsten Patch wollen wir folgendes Verhalten umsetzen (d.h. in „vuvu“ implementieren): Wir drehen die Maus an einer beliebigen Position am Schreibtisch um (siehe Abbildung auf der nächsten Seite), bewegen die umgedrehte Maus an den „linken Rand“ (das Mausexploration liegt dabei kontinuierlich auf der Schreibtischplatte auf) und drücken eine Taste auf der Tastatur um diesen Punkt als den linken Rand zu markieren. Danach „fahren“ wir mit der Maus an den „rechten Rand“ (Mausexploration liegt wiederum ständig auf und bewegt sich damit) und drücken eine weitere Taste auf der Tastatur um diesen Punkt als den größten Wert zu markieren (damit legen wir gleichzeitig den rechten Rand im Explorer-Fenster fest).





Vorgangsweise

- Wir starten mit dem Patch „Mausrad2.v4p“ und speichern ihn als „Mausrad3.v4p“ ab.
 - Die Basisidee, die wir zunächst umsetzen, ist es, den Bereich zwischen linkem und rechtem Rand (sowohl am Schreibtisch als auch im Renderer-Fenster) jeweils auf den Wertebereich -1.0 bis +1.0 zu mappen. Dazu führen wir folgende Änderungen bzw. Ergänzungen im Patch durch:
- (1) Zunächst entfernen wir den Divisions-Node („/ (Value)“) und die daran angeschlossene „IOBox (Value Advanced)“ ein und fügen einen neuen „Map (Value)“-Node ein. Diesen vergrößern/verbreitern wir, um in der Folge die einzelnen Eingabepins leichter erreichen zu können. Den Zwischenstand zeigt die Abbildung oben.
- (2) Node-Typ „IOBox (Value Advanced)“, „Doppelklick links“ und Wert „-1.0“ eingeben, Beschriftung „Ausgabe min.“ im „Herrn Inspektor“ hinzufügen (Attribut: „Descriptive Name“)
- (3) Node-Typ „IOBox (Value Advanced)“, „Doppelklick links“ und Wert „1.0“ eingeben, Beschriftung „Ausgabe max.“ im „Herrn Inspektor“ hinzufügen (Attribut: „Descriptive Name“)



(4) Node-Typ „IOBox (Value Advanced)“, mit dem Ausgabe-Pin von „Map (Value)“ verbinden, im „Herrn Inspektor“ das Attribut Size auf „20.0 px“ setzen. Dieser Node dient uns zur Kontrolle ob die Skalierung korrekt funktioniert

(5) Einfügen eines Nodes vom Typ „Keyboard (System Desktop)“ sowie folgender Nodes:

(6) Node-Typ „KeyMatch (Keyboard)“ und verbinden des Output-Pins von (5) mit dem Eingabe-Pin dieses Nodes (Überprüfung auf Übereinstimmung mit zu definierenden Tasten). Die Tasten die zu überwachen/evaluieren sind, sind im „Herrn Inspektor“, durch Beistriche getrennt, beim Attribut „Key Match“ zu spezifizieren. Wir verwenden als Tasten die linke und rechte Pfeiltaste und geben dazu in Key Match: „Left, Right“ ein. Der Node „KeyMatch (Keyboard)“ bekommt daraufhin zwei Ausgabe-Pins für die beiden zu überwachenden Tasten

(7) Einfügen eines Nodes vom Typ „IOBox (Enumerations)“, verbinden des Output-Pins mit dem Eingabe-Pin „Key Mode“ von (6), Festlegen des Werts (Drop-Down Menü) auf „UpOnly“ (Keyboard-Tasten werden nur beim Auslassen ausgewertet)

(8) Einfügen von zwei „IOBox (Value Advanced)“-Nodes; Value Type: „Boolean“, Behavior: „Toggle“ und verbinden mit „KeyMatch (Keyboard)“ (6) wie in obigem „Schaltplan“ dargestellt

(9) Einfügen von zwei „S+H (Animation)“-Nodes und Erstellen der Verbindungen wie oben dargestellt

- Damit wir im Ausgabefenster in weiterer Folge kein Quadrat mehr sehen, sondern die Position des Mausekzes als eine Art „Fadenkreuz“, wählen wir den Node „Transform (Transform 2D)“ aus und stellen im „Herrn Inspektor“ folgendes ein: ScaleX: „0.0“, ScaleY: „2.0“; in der Dropdown-Liste des Nodes „IOBox (Enumerations)“ wählen wir „WireFrame“ (anstelle von „Solid“) aus
- Patch abspeichern
- Nun können Sie den fertigen Patch testen:
 - Machen Sie sich 2 Markierungen auf dem Schreibtisch, z. B. mit Postlts – bitte nicht mit Bleistift, schon gar nicht mit Faserschreiber ;-)
 - Klicken Sie am Bildschirm in das Renderer-Fenster
 - Drehen Sie die Maus um, fahren Sie mit dem Mausekz an Ihre linke Markierung, drücken sie die „Pfeiltaste links“, fahren Sie mit dem Mausekz bis zur rechten Markierung und drücken Sie die „Pfeiltaste rechts“
 - Nun müsste der Bereich linke bis rechte Markierung am Tisch genau auf das Renderer-Fenster gemappt sein...
- **Frage 9:** Welches unschöne Problem stellen Sie fest? Was könnte man dagegen tun?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.

2.c) Alternative Eingabegeräte: Joystick, „Leap Motion“, „Kinect“



Mit dem Patch „Mausekz3.v4p“ haben wir jetzt gezeigt, wie man die Maus (bzw. das Mausekz) als linearen Positionsgeber verwenden kann. Das funktioniert zwar grundsätzlich sehr gut, ist aber wohl etwas „uncool“. Kein Problem – in „vvvv“ ist es sehr einfach, die Maus durch alternative Eingabesensorik wie etwa Joysticks oder Gestenhardware (Leap Motion, Kinect) zu ersetzen. Wie das geht, sehen wir uns im Folgenden (kurz) an. Der Ersatz der Mausekzsteuerung durch Gesten oder Joystickbewegungen sollte, dank bereits verfügbarer Nodes in „vvvv“, pro Fall nicht länger als 5 Minuten dauern (für „visuelle Programmierer“ mit Grunderfahrung in der Benutzung von „vvvv“...).

- Ausgangspunkt ist der Patch „Mausekz3.v4p“, von dem wir (im Windows Explorer) drei Kopien erstellen: „MausekzJoystick.v4p“, „MausekzLeap.v4p“, und „MausekzKinect.v4p“.
- Wir öffnen den ersten Patch (z. B. „MausekzJoystick.v4p“) und ersetzen den „Mouse (Devices Desktop/Windows)“-Node durch einen passenden Node für Joystick, Leap Motion, Kinect, etc. Beachten Sie, dass diese Nodes unter

Umständen die X- bzw. Y-Koordinaten in anderer Form an den Ausgabe-Pins bereitstellen, als wir es vom „Mouse“-Node kennen...

- ◦ **Joystick:** „GameController (Devices Analog)“, Sehen Sie sich die Konfigurationsmöglichkeiten und Output-Pins an und wählen Sie einen „passenden“ Output-Pin zur Steuerung der Position im Renderfenster
- ◦ **Leap Motion:** „Leap (Devices)“. Liefert am ersten Ausgabe-Pin die Position der Hand als ein Spread von X-, Y-, und Z-Koordinaten („*HandPositionXYZ*“). Nachdem wir hier nur die X-Koordinate (links/rechts-Bewegungen) benötigen, ist dieser mittels eines „GetSlice (Spreads)“ herauszulösen und an den Patch weiterzugeben...
- ◦ **Kinect:** Die Kinect liefert eine Fülle an Daten und kann auf mehrere Arten eingebunden werden. Am besten ist es, mit dem Node „Kinect (Devices Microsoft DX 9)“ zu starten und dessen ersten Output-Pin („*Kinect Runtime*“) als Eingabe für einen „Skeleton (Kinect Microsoft DX 9)“-Node zu verwenden. Dieser liefert Bewegungen, die von einer Kinect erkannt werden (z. B. von Hand, Körper) in XYZ. Wie bereits zuvor beim „Leap (Devices)“-Node ist auch hier wieder die X-Koordinate „herauszulösen“... Damit das „Skeletal Tracking“ funktionieren kann, sind im „Kinect (Devices Microsoft DX 9)“-Node dazu die Eingabe-Pins „Enabled“ sowie „Enabled Skeleton“ auf 1 zu setzen.
- Versuchen Sie, mit diesen Informationen die ursprüngliche Funktion des Patches (linearer Positionsgeber mit dem Mousrad) mit einem Joystick, einer Leap Motion bzw. einer Kinect zu implementieren. Wie Sie feststellen werden, haben wir durch die Trennung des Wertebereiches von Ein- und Ausgabe bereits zuvor sichergestellt, dass die Integration dieser Sensoren ohne sonstige Programmänderungen auskommt. Konfiguration/Einstellung von linkem bzw. rechtem Rand kann genauso wie zuvor für die Maus mit der Pfeiltaste links bzw. rechts vorgenommen werden.
- **Frage 10:** Berichten Sie über Ihre Erfahrungen, Schwierigkeiten bzw. auch „A-ha“-Effekten, die Sie mit dieser Aufgabe gemacht haben; *dokumentieren Sie im Protokoll Ihre Patch-Änderungen durch Screenshots bzw. geben Sie die modifizierten Patches (in Moodle) ab.*

3) Eine sinnvolle Anwendung – „Points of Interest“ in einem Foto

In dieser Aufgabe verwenden wir den gerade eben erstellten Patch „Mausrad3.v4p“ in einem größeren Kontext, um damit die Position in einem Foto zu steuern um „Points of Interest“ (POIs) anzufahren und sich dazu Detailinformationen anzeigen zu lassen. Natürlich können Sie hierzu auch die modifizierten Versionen des Patches „Mausrad3.v4p“ verwenden, um die Ansteuerung über Joystick, Leap Motion, Kinect, etc. vorzunehmen. In der Folge betrachten wir hier nur die Basisvariante mit der Mausesteuerung.

3.a) Vorbereitung: Kennenlernen des Projektes „PointsOfInterest“

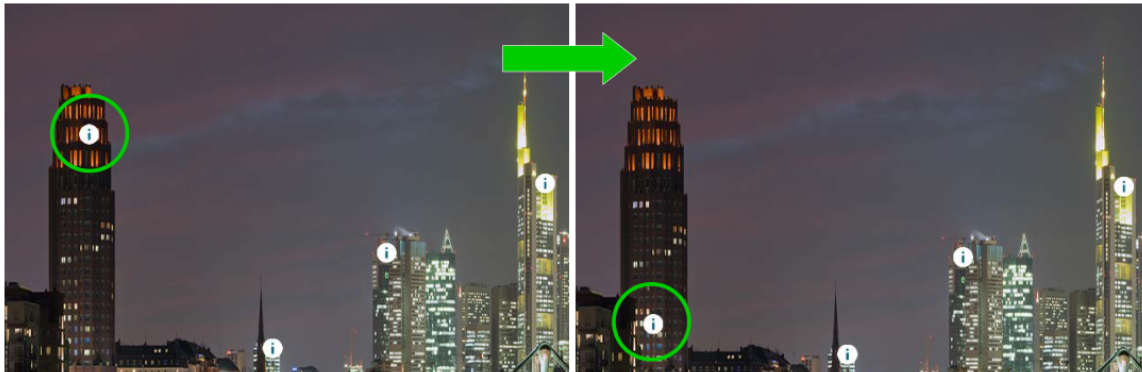
Laden Sie sich dazu den Ordner „PointsOfInterest“ vom Laufwerk „P:“ (Datentausch) und öffnen Sie den Patch mit dem Dateinamen „_root.v4p“. Wie Sie feststellen, ist dieser Patch sozusagen das „Hauptprogramm“ und greift auf andere Patches („Hilfsklassen“) zu, unter anderem auch auf den gerade erstellten Patch „Mausrad3.v4p“ (sie finden hier eine Art „Musterlösung“ dafür), sowie auf „GUI.v4p“ bzw. „Accordion.v4p“.

Machen Sie sich vertraut mit diesem „vvvv“-Programm und den darin verwendeten Subpatches und beantworten Sie folgende Fragen:

- **Frage 11:** Wofür steht der Parameter „Epsilon“ in „_root.v4p“? Was passiert, wenn Sie diesen auf 0.01 bzw. auf 0.3 ändern?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.

- Versuchen Sie das „i in weißem Kreis“ (Zusatzinformation zu einem „Point of Interest“) für den „Main Plaza“ (=linker beleuchteter Turm im Bild) von dessen Spitze irgendwo im Bereich des 8. Stockwerkes zu verschieben – wie nachfolgend dargestellt/demonstriert.



Frage 12: Was mussten Sie dazu am Patch ändern?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll.

3.b) Anpassen des Patches „PointsOfInterest“ für eine eigene Applikation

Der nächste Schritt ist es nun, dieses „vvvv“-Programm als Basis für eine ähnliche Applikation zu verwenden und entsprechend anzupassen. Im Datenaustausch-Laufwerk „P:“ finden Sie einen Dateiordner „assetsTHI“ den Sie in das Projektverzeichnis von „PointsOfInterest“ kopieren. Sehen Sie sich die Dateien dieses Ordners, sowie auch des ursprünglichen Ordners „assets“ an und vergleichen Sie die Inhalte. „assetsTHI“ enthält u.a. ein Foto des Hochschulcampus, das Sie nun anstatt der Skyline von Frankfurt als Basis einbetten sollen. Weiteres sind in diesem Ordner bereits Grafiken für die „Hot Spots“ bzw. „Points of Interest“ enthalten, die Sie ebenfalls (korrekt) platzieren und verlinken sollen. Folgende POIs sind zu verwenden: 1) Kavalier Dallwigk, 2) Audi Akademie, 3) Gießereihalle, 4) Hochschule, D, 5) Hochschule, G, 6) CARISSMA, 7) THI-Erweiterung.

Das Ergebnis nach Bearbeitung sollte in etwa so aussehen:



- **Frage 13:** Was mussten Sie alles ändern, um zu dieser Lösung zu gelangen? Gab es Schwierigkeiten – welche? Dokumentieren Sie Ihre Lösung bzw. den Lösungsweg textuell, machen Sie Screenshots (und betten diese in Ihrem Protokoll ein). Geben Sie den gesamten Dateiordner inkl. aller Patches, Subpatches, Assets in Moodle ab).

FEEDBACK zu Aufgabenblatt 2

Sie können den Feedbackbogen gerne abtrennen und anonym abgeben (z. B. zusammengefaltet am Ende eines Praktikumstermins, in der Vorlesung oder in mein Postfach beim I-Dekanat im B-Gebäude, EG). Vielen Dank!

Bitte füllen Sie diese Fragen (**freiwillig**) aus um die Veranstaltung zu verbessern.

- Das behandelte **Thema passt für mich/uns zum Inhalt/Modulbeschreibung der Lehrveranstaltung** (d.h. keine „Themenverfehlung“)...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die **technische Tiefe/Komplexität** des Aufgabenblattes war **zu hoch**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Dieses Aufgabenblatt hat mir/hat uns **Spaß** gemacht...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Ich/wir haben etwas **gelernt**, möchten das **in Zukunft evtl. selbst anwenden**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die Aufgabenstellungen waren **einfach genug** um von mir/uns **verstanden** zu werden...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die **Zusatz-/Hintergrundinformationen** auf dem Aufgabenblatt waren für mich/uns **nützlich**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Verbesserungsmöglichkeiten? ...können Sie in diesem Feld angeben falls Sie möchten

Feedback/Kritik wird selbstverständlich vertraulich behandelt und in keiner Weise zur Bewertung der Leistung des Studenten/der Gruppe herangezogen!